



CS240 Algorithm Design and Analysis

Final Review

2026.06.18



Mergesort



- **Mergesort**

- Divide array into two halves
- Recursively sort each half
- Merge two halves to make sorted whole

input

A	L	G	O	R	I	T	H	M	S
---	---	---	---	---	---	---	---	---	---

sort left half

A	G	L	O	R	I	T	H	M	S
---	---	---	---	---	---	---	---	---	---

sort right half

A	G	L	O	R	H	I	M	S	T
---	---	---	---	---	---	---	---	---	---

merge results

A	G	H	I	L	M	O	R	S	T
---	---	---	---	---	---	---	---	---	---

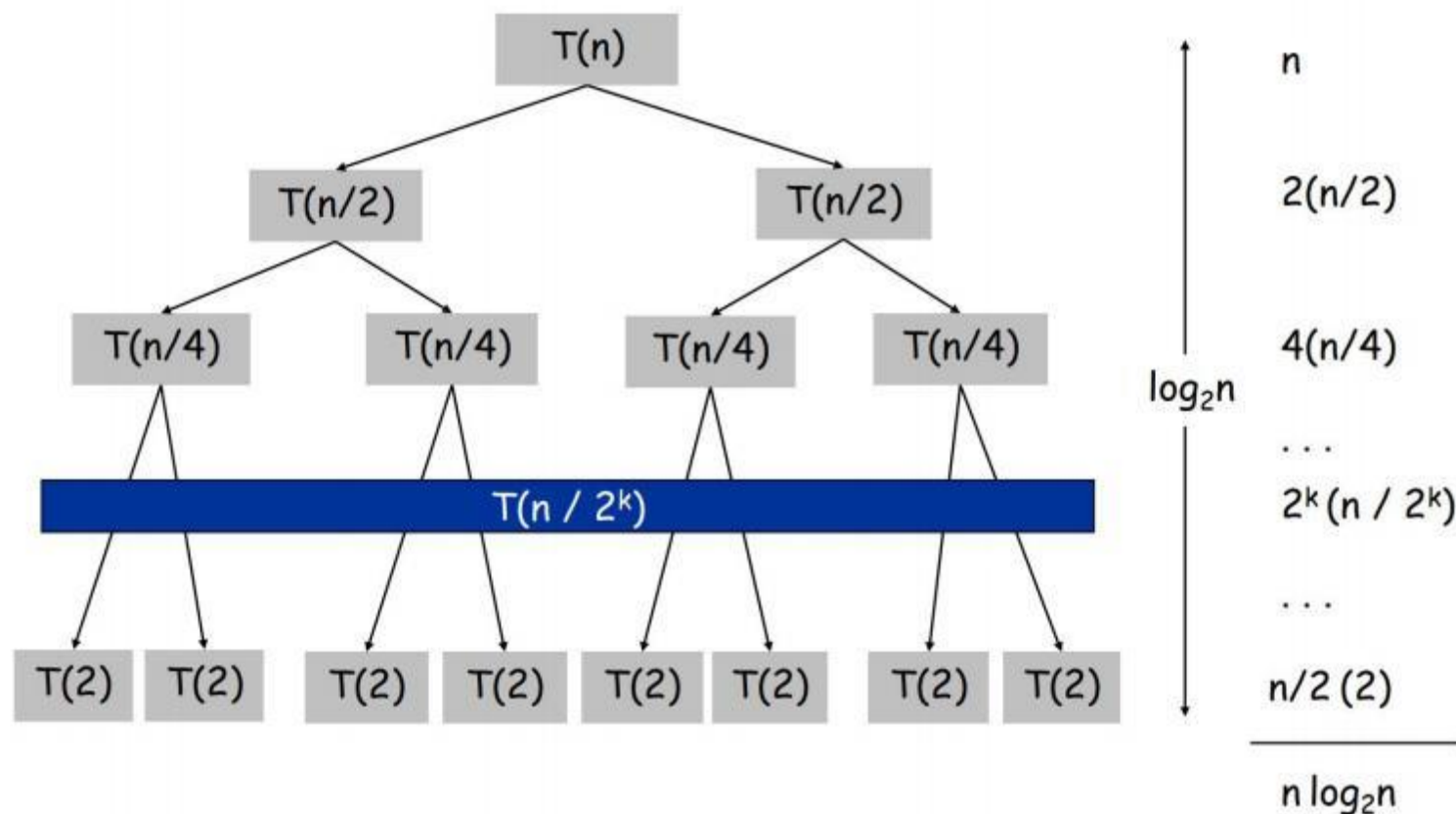




Proof by Recursive Tree



$$T(n) = \begin{cases} 0 & \text{if } n=1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$





Proof by Recursive Tree

Layer	# sub problems	subproblem size	total cost at this level
0	1	n	n
1	2	n/2	n
2	4	n/4	n
...	...	...	...
log n	n	1	n

$$f(n) = n \Rightarrow f(n/2^i) = n/2^i$$

$$2^i \cdot (n/2^i) = n$$

$$T(n) = \sum_{\text{level } i} (\text{cost at level } i)$$

$$T(n) = n \cdot \log n$$



Proof by Master Theorem

$$T(n) = aT(n/b) + f(n)$$

$$a = 2, b = 2, f(n) = n$$

$$n^{\log_b a} = n^{\log_2 2} = n$$



0

1 sub problem

size : n

$$\frac{n}{b^k} = 1$$

$$f(n) = n$$

$$T(n) = \Theta(n^{\log_b a} \log n)$$



1

a sub problems

size : n/b

$$n = b^k$$

and

$$T(n) = \Theta(n \log n)$$

$$n^{\log_b a} = n$$



2

a^2 sub problems

size : n/b^2

$$a^k = a^{\log_b n}$$

$$a^{\log_b n} = n^{\log_b a}$$



k

a^k sub problems

size : n/b^k



Master theorem

Step 1. Compute the baseline

- $n^{\log_b a}$ ← *this is the recursion work*

Step 2. Compare $f(n)$ with this baseline

Case 1

$$n^{\log_b a}$$

Case 2

$$n^{\log_b a} \log n$$

Case 3

$$f(n)$$

Case 1 — recursion is heavier (bottom dominates)

- The recursive work is bigger than the extra work.
- The tree grows a lot at the bottom, so most work is in the leaves.

Case 2 — balanced (exact match (this is strict!))

- The recursive work and extra work are about the same.

Case 3 — extra work is heavier (top dominates, $f(n)$ is larger!)

- The top levels already do a lot of work, so recursion doesn't matter much.

Ex:

- n vs $n \log n \rightarrow$ **NOT equal**
- n vs $n^2 \rightarrow$ **Case 3**
- n vs $1 \rightarrow$ **Case 1**



Geometric Distribution Intuition



Repeated Independent Trials Until First Success

Suppose each trial succeeds with probability p , independently of all previous trials.

Let R be the number of trials until the first success.

Then R follows a geometric distribution.

Example 1: Coin Flipping

A coin lands heads with probability

$$p = \frac{1}{2}.$$

We repeatedly flip the coin until we see the first head.

Question:

How many flips do we expect to make?

Since each flip succeeds with probability $p = 1/2$,

$$E[R] = \frac{1}{p} = 2.$$

Interpretation:

On average, we need about 2 flips to see the first head.

$$p=1/2.$$

$$P(R = 1) = \frac{1}{2}$$

$$P(R = 2) = \frac{1}{2} \cdot \frac{1}{2}$$

$$P(R = 3) = \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2}$$

...

$$P(R = k) = (1 - p)^{k-1} p.$$

$$E[R] = \sum_{k=1}^{\infty} k P(R = k).$$

$$E[R] = \sum_{k=1}^{\infty} k (1 - p)^{k-1} p.$$





Geometric Distribution Intuition



Example 2: Rolling a Die

A fair die is rolled repeatedly.

Success = rolling a 6.

$$p = \frac{1}{6}.$$

Therefore,

$$E[R] = \frac{1}{p} = 6.$$

Interpretation:

On average, we need 6 rolls to see the first 6.

Example 3: Randomized Algorithm

Suppose a randomized algorithm succeeds with probability

$$p = 0.1$$

in each independent run.

We repeatedly restart the algorithm until it succeeds.

Then

$$E[R] = \frac{1}{0.1} = 10.$$

Interpretation: The algorithm succeeds after about 10 runs on average.





Why is $E[R] = 1/p$?



Let

$$E[R] = E.$$

The first trial: succeeds with probability p , fails with probability $1 - p$

Therefore,

$$E = p \cdot 1 + (1 - p)(1 + E).$$

Expanding,

$$E = 1 + (1 - p)E.$$

Hence,

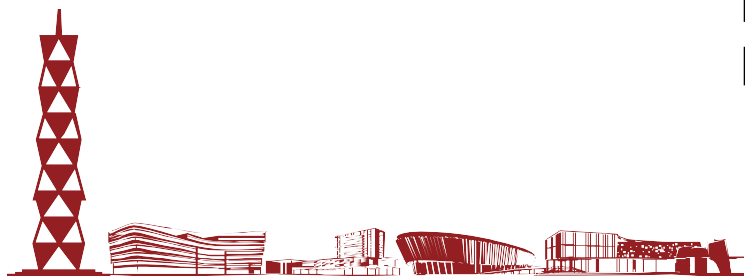
$$pE = 1.$$

Therefore,

$$\boxed{E[R] = \frac{1}{p}}.$$

Probability: $P(R = k) = (1 - p)^{k-1}p.$

Expectation: $E[R] = 1/p$

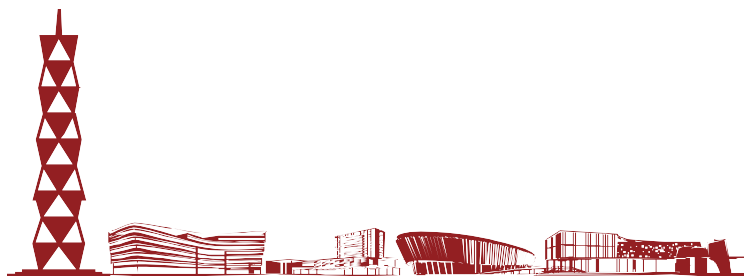




Approximation Ratio



- Let X be a maximization problem. Let A be an algorithm for X .
- Let $a > 1$ be a constant.
- A is an a -approximation algorithm for X if A always returns an answer that's at least $1/a$ times the optimal.
 - **Ex** If X is max-flow, A is a 2-approx algorithm if it always returns a flow that's at least $1/2$ the optimal.
 - The closer a is to 1, the better the approximation.
- If X is a minimization problem, A is an a -approximation algorithm for X if it always returns an answer that's at most a times larger than the optimal.
 - **Ex** If X is min-cut, A is a 2-approx algorithm if it always returns a cut that's at most 2 times the size of the optimal.
 - Again, the closer a is to 1, the better the approximation.



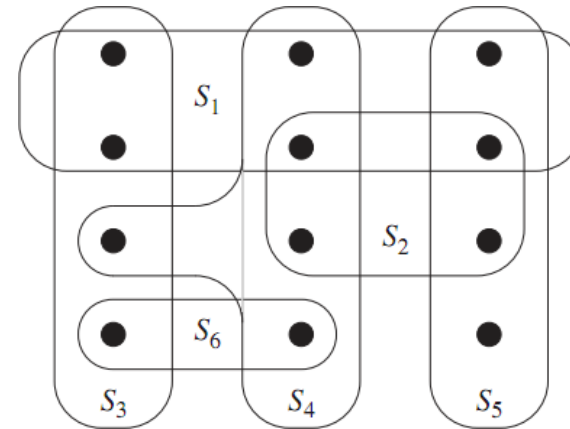


Set Covering



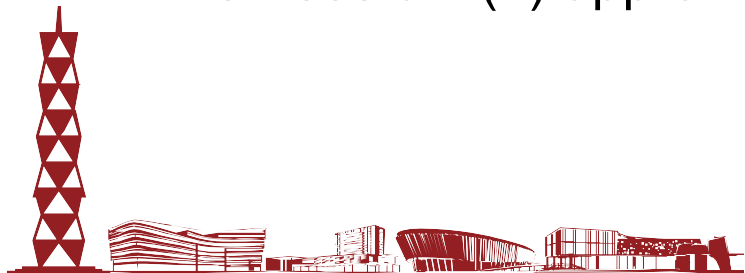
- **Input** A collection F of sets. Each set has a cost. The union of all the sets is X .
- **Output** A subset G of F , whose union is X .
- **Goal** Minimize the total cost of the sets in G .

$$\min \sum_{S \in G} \text{cost}(S)$$



If all sets have same cost, S_3 , S_4 and S_5 is a min cost set cover of X .

- Minimum cost set cover is NP-complete.
- We'll see a $\ln(n)$ -approximation algorithm, where $n=|X|$.

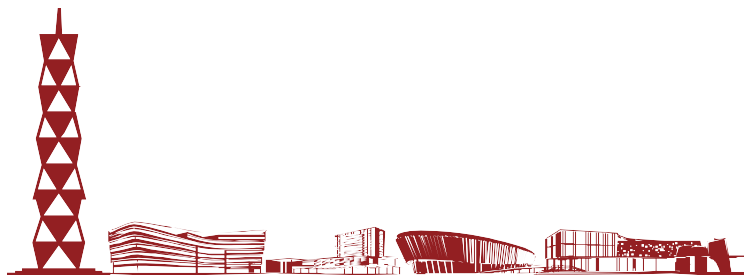




A Greedy Approximation Algorithm



- A natural greedy heuristic is to choose sets which cover points most cheaply.
 - For each set, let c be its cost, and m be the number of points it covers.
 - We want to use the set with the smallest c/m value, because this is the cheapest way to cover some new points.
- After we pick this set, remove all the points it covers. Then we consider the per unit cost of the remaining sets and again choose the cheapest.

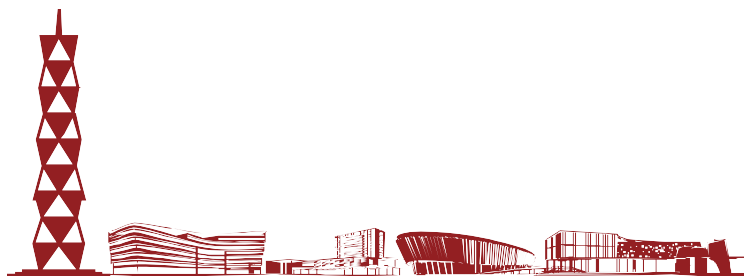




A Greedy Approximation Algorithm



- F is the entire collection of sets. The union of these sets is X .
 - Each set S in F has a cost $\text{cost}(S)$.
 - U is the set of elements of X we haven't covered yet.
 - C is the set cover we eventually output.
- $U = X$
 - $C = \emptyset$
 - while $U \neq \emptyset$
 - choose $S \in F - C$ with min $|\text{cost}(S)|/|S \cap U|$
 - $C = C \cup \{S\}$
 - $U = U - S$
 - output C
- Per unit cost to cover new elements.





Proof of Correctness



- We always output a set cover, because the while loop continues till X is covered.
- We'll prove the approximation ratio is at most $1 + 1/2 + 1/3 + \dots + 1/n \approx \ln(n)$.
- $\text{OPT} = V$. The first element is charged **at most** V/n , The second element is charged **at most** $\frac{V}{(n-1)}$, For the j -th element covered by the greedy algorithm, its charge is at most $\frac{V}{n-j+1}$,
- If the min cost of a set cover is V , our set cover costs at most $\ln(n) \cdot V$.
- The basic plan is to bound the cost of the set cover the algorithm outputs using the “average cost” per element.

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \quad H_n = \sum_{i=1}^n \frac{1}{i}$$

$$\frac{\text{Greedy Cost}}{\text{OPT}} \leq \ln n.$$

$$H_n = \Theta(\ln n).$$

